Synchronization and Consistency

Tom Conte, Brian Railing

Copyright © 2012: Thomas M. Conte
These notes MAY NOT BE REPRODUCED WITHOUT EXPRESS PERMISSION OF THE AUTHORS

1

The disadvantage of shared memory

- Shared memory is a blessing in that it allows parallel, simultaneous access to data by multiple threads
  - Not possible in message passing
- It's a curse because allowing this means two threads need to coordinate their actions

2

Types of synchronization

- Generally, there are two general types of coordination that programmers require:
- Barriers
  - makes sure all threads are at the same place before proceeding
  - Note that not all threads in a program have to "participate" in a given barrier
- Mutex
  - Makes sure only one thread can access a critical region
- Turns out they can each use the same underlying infrastructure

SYNCH-3

3

Primitives vs. programming models

- Generally, programmers use mutex via an API (e.g., pthreads)
- The API itself is implemented using "primitives"
  - Either an OS function
  - or a hardware feature
- But note: the primitives used by a model may themselves be models that use other primitives, etc
- No clear/hard/fast definition of which is which

SYNCH-4

4

Mutex synchronization

- getlock(lock)
  - Blocks until "lock" is acquired
  - Guarantees all other threads waiting on "lock" must wait
  - unlock(lock) lets next thread go
- Code:

```
getlock(lock)
// critical section goes here
unlock(lock)
```

SYNCH-5

5

Implementing getlock()

- use test and set

TS(x) *means*

atomically set location x to a 1 and return the prior value stored at x

- Using TS to implement getlock/unlock:

```
getlock(x) {
      while (TS(x) == 1)  // a "spinlock"
            ;
}

unlock(x) { x = 0; }
```

SYNCH-6

6

Implementing TS with atomic swap:

- test and set

```
TS(int x) {
      oldval = SWAP(x,1); // atomic swap
      return oldval;
}
```

Architecture must provide for an atomic SWAP instruction that swaps memory with a value, returning the prior value of memory

SYNCH-7

7

Problems with TS

- When processor *i* requests the line that holds the lock, it performs a GetM
- Each processor then is performing GetMs
- This results in a huge amount of either invalidations or cache to cache transfers
- But NO ONE is changing the value of the lock (overwriting it with a "1")
- Better if every processor is performing GetS…

SYNCH-8

8

Test-and-Test-and-Set (TTS)

- One alternative is to perform Test and Test and Set:

```
TTS(int x) {
    oldval = x; // the read of x
    if (oldval == 0) oldval = SWAP(x,1);
    return oldval;
}
```

Use in lock, same as before…

```
while (TTS(lock) == 1)
        ;
```

- Good: All threads are doing GetS instead of GetM
- Bad: There is still a rush of invalidations when SWAP does occur (and lock is set to 0)

SYNCH-9

9

Fetch and Op

- New synchronization primitive
- Place special purpose hardware at memory unit to do the following atomically

```
fetchAndOp (x, c) {
        return(x = x op c);  // this step is atomic
                             // example op is '+'
}
```

TS or TTS can also be used to implement Fetch and Op (Q: how? A: work it out!)

SYNCH-10

10

Safe Barrier Synchronization

```
// On startup, initially
counter = 0;
toggle = 0;
release = !toggle;
...
barrier(int N) {
  if (FetchAndAdd(counter, 1) < N)
  while (release != toggle)
          ;      // spin to be released
  else {
    counter = 0;                // Reset counter
    release = toggle;    // Release threads
     toggle = !toggle;  // set up for next time
  }
}
```

- This allows the barrier code to be re-used w/o resetting "release" to zero

SYNCH-11

11

Barrier support in hardware

- Use "1-hot" id for each processor
  - e.g., for N = 4, P0 is 0001, P1 is 0010, P2 is 0100, P3 is 1000
- Have a global wired-OR bus with N lines for N processors
  - (A wired-OR bus is one that multiple pro
- Algorithm:
  - Processor arrives at barrier and drives the processor's 1-hot id onto bus
  - Spin until bus shows all 111…1

SYNCH-12

12

Parallel FOR loops using F&A

#pragma parallel-for

```
for (i = 0; i < N; i++) {
        Loop body using i
}
```

Can be implemented as

```
fork(); // fork K worker threads
// each thread:
while ((iLocal = fetchAndAdd(i)) !=N)
        Loop body using i = iLocal
join();
```

SYNCH-13

13

Another synch primitive—LL/SC

LL = "Load Linked"

- **Load Linked loads a location and watches the location ("link"-- terrible name for this, but we're stuck w/it)**

  LL R1,addr means

  R1 = Mem[*addr*];

  ***link* = *addr*; // 'link' is internal register**

  If $ block for addr is invalidated, then link register is set to 0

SYNCH-14

14

LL/SC continued

SC = "Store Conditional"

- **Store conditional ONLY updates memory if the value ("link") hasn't changed, and returns success/failure**

```
SC R1,addr means
        if (link == addr) then {
           Mem[addr] = R1;
           R1 = 1;
        } else
           R1 = 0;
```

Link check is easy to implement

SYNCH-15

15

lock(x) with LL/SC

```
lock (x)
    do {
        R2 = 1;
        LL R1,X         // load link from addr X, set link register to X
        SC R2,X         // Try to store a 1 if Link == X
    } while (R1 != 0 || R2 == 0);
```

SYNCH-16

16

An alternative to the madness…

- Get rid of locks entirely– use *Transactions*
- Idea: processor X begins a transaction by checkpointing memory
  - It modifies whatever it wants.
  - When the transaction ends, it checks for any conflicts (ie, what it modified != what is there now)
  - If conflict, retry transaction
- Can be implemented in software with huge cost
  - Helped if the L1$ can be locked so it does not update main memory until end of transaction, but not enough…
- Hardware support is an active research area

SYNCH-17

17

Intel Haswell's TSX

- XBEGIN / XEND
  - Enter / leave a transactional region
  - XBEGIN specifies cache blocks that are "transactional"
  - The tracking granularity is explicitly a cache block
- Implementation
  - If coherence requires a state change in a transactional block, then abort
  - Then clear flags when transaction completes

SYNCH-18

18

A problem…

- Coherence is not enough…

Program order

```
PE 1        PE 2
D = 1       FF <= F
F = 1       DD <= D
```



Execution order

```
PE 1        PE 2
D = 1       DD <= D
F = 1       FF <= F
```

PE 2 can reorder loads

- Possible outcomes on PE 2 in program order
  - (FF,DD) can be (0, 0), (0,1), (1, 1)
  - Out of order execution can also give (1,0)
- The bad…
  - Breaks things

SYNCH-19

19

Memory Consistency

- Why would we want to write code like that?
- Example: flag to guard a critical section

```
flag = 0
X = 2
Y = 4

PE 1:                PE 2:
X = Y;               while (flag != 1)
flag = 1                 ;
                     Z = X; // can be either 2 or 4!
```

- If (1) (access to Y) misses, PE 1 will reorder accesses and performs (**flag = 1**) assignment first
  - PE 2 gets the old value of X (d'oh!)

SYNCH-20

20

Memory Consistency

- Coherence is about ordering of parallel accesses to the same address
- Consistency is about ordering of parallel accesses between different addresses

SYNCH-21

21

Strict Consistency

- The result of any execution should be the same as if
  1. The accesses executed by each PE were kept in-order (no read or write reordering), and
  2. The accesses between different PEs are ordered by a global time clock, and
  3. The same ordering is seen by all PEs

Example if by global clock…

PE1: R(x)==0……..R(x)==1

PE2: …….W(x)=1

Then this observation of another PE can't happen:

PE1: ……..R(x)==0….R(x)==1

PE2: W(x)=1

SYNCH-22

22

Sequential Consistency

- The result of any execution should be the same as if
  1. The accesses executed by each PE were kept in-order (no read or write reordering), and
  2. The accesses among different PEs were arbitrarily interleaved (eg, such as by an OS), and
  3. The same interleaved order is seen by all PEs
- So this scenario is now valid….

PE1: ……..R(x)==0….R(x)==1

PE2: W(x)=1

SYNCH-23

23

Sequential Consistency

- Simple implementation.
  - A PE issues next access only when all previous accesses (from it or all other PEs) are complete
  - Sloooow!
- A better implementation
  - PE issues accesses as it sees fit, but *detects and fixes potential violations of sequential consistency*
  - *How? Beyond the scope of this course… but EXPENSIVE*

SYNCH-24

24

Relaxed Consistency Models

- Data Races
  - When data accesses to same variable in different PEs are not protected by locks
  - Most well-written programs are data-race-free via mutex, barriers, etc.
  - This allows relaxed consistency models
- Kinds of relaxed consistency:
  - Weak Consistency, Release Consistency, Processor Consistency, Lazy Release Consistency, Scope, etc, etc [we cover the first one]
  - When there are data races, the more relaxed the model, the weirder the program behavior!

SYNCH-26

26

Example: Weak Consistency

1. Accesses to locks (AQ or R) are sequentially consistent
2. All accesses in the whole system must complete before a lock access (flush on AQ or R)
3. AQ, R take time: No shared data access (read or write) by any PE are allowed until all pending synchronization accesses have completed (i.e., lock is acquired or released)

SYNCH-28

28

Weak: good and bad

- Why is weak consistency good?
  - Can reorder at will between AQ/R: Don't have to broadcast changes *at all* until the synchronization accesses take place
  - …this reduces the amount of communication needed
- Why is it bad?
  - all global reads/writes before a AQ/R must update ***everyone***

  But we only "care" about accesses that are "guarded" by synchronization (ie, the critical section variables), so…

SYNCH-29

29

Consistency Lessons Learned

- Performance vs. Complexity vs. Programmability
  - Easier to write and debug with stronger consistency
  - Easier to get performance with weaker consistency
  - Can add hardware to close performance gap
- Good old days:
  - Hardware was expensive
  - Expert programmers: can use weaker consistency
  - Result: weaker consistency invented (and used)
- Now: cheap HW, lots of different programmers

SYNCH-30

30